

Team and Responsibilities

Name	Khaytboy Khayrullaev
TP number	TP072305
Email	TP072305@mail.apu.edu.my
Module	CT050-3-2-WAPP Web Applications
Assessment	Group assignment, carried out individually
Role	Sole developer: every area of the application, its database, tests, documentation and deployment

The brief describes a group assignment, and this submission was completed by one member working alone. Every part of the system — requirements, database design, application code, tests, documentation and deployment — was designed, built and verified by the author above. The sections that follow describe what that involved, so the marker can see the full scope of the work rather than a divided set of responsibilities.

1. Areas owned

Every area below was the responsibility of the single member.

Area	What it covers
Requirements and proposal	Interpreting the brief, audience modelling, scope, requirement list and schedule
Architecture	Solution structure, layering, configuration, dependency injection and the request pipeline
Database	Entity model, EF Core configuration, migrations, constraints, indexes and the ERD
Authentication and accounts	ASP.NET Core Identity, roles, registration, login, profile and password change
Public site	Home page, About, Contact, Privacy, the course catalogue, search, filters and paging
Student area	Dashboard, My Courses, enrolment, lessons, quizzes, results and progress tracking
Learning content	Lessons as articles, videos, PDFs, images and exercises; validated uploads and private file streaming
Quizzes	Quiz, question and answer management; points-based grading; attempt history and review
Administration	Admin dashboard plus CRUD for courses, categories, resources, quizzes, questions, answers, users and enrolments
Design system	Colour system, typography, components, responsive layout and accessibility
Security	Authorisation, antiforgery, security headers, rate limiting, upload validation and the security review

Area	What it covers
Testing	Unit, service, data and integration tests; Playwright browser tests; the smoke test
Deployment	Dockerfile, Railway hosting with a persistent volume, Firebase Hosting for the presentation site
Documentation	All eighteen documents in <code>docs/</code> , the README, the ERD, the wireframes and the PDF build
Viva preparation	Preparing explanations for every part of the system

2. Files owned

Paths are relative to the repository root. With one member the whole repository is in scope; the list below groups it by area so it is clear what was built.

Application and configuration

- `src/LearnHub/Program.cs`, `src/LearnHub/appsettings.json`, `LearnHub.csproj`, `Directory.Build.props`, `Directory.Packages.props`, `global.json`
- `src/LearnHub/Infrastructure/` — middleware, options, tag helpers, validation attributes, claims helpers and the interchange-map renderer
- `src/LearnHub/Controllers/` — Home, Courses, Categories, Account, Profile, Quizzes, Resources, Student, Error
- `src/LearnHub/Areas/Admin/` — the administration controllers and views

Data

- `src/LearnHub/Models/`, `src/LearnHub/Data/ApplicationDbContext.cs`
- `src/LearnHub/Data/Configurations/` — entity configuration, indexes, check constraints and delete rules
- `src/LearnHub/Data/Migrations/` — the single `InitialCreate` migration
- `src/LearnHub/Data/Seed/` — the demonstration catalogue, the seeder and the development password generator

Services

- `src/LearnHub/Services/` — catalogue, category, enrolment, dashboard, progress, resource, quiz, grading, management, user, contact and lookup services
- `src/LearnHub/Services/Storage/` and `src/LearnHub/Services/Content/` — upload rules, file storage, video parsing and lesson rendering

Interface

- `src/LearnHub/ViewModels/`, `src/LearnHub/Views/`, `src/LearnHub/Areas/Admin/Views/`
- `src/LearnHub/wwwroot/` — design system CSS, admin CSS, JavaScript, course artwork and vendored libraries

Tests

- `tests/LearnHub.Tests/` — unit, service, data and integration tests
- `tests/e2e/` — Playwright browser tests across desktop, tablet and mobile

Delivery and documentation

- `Dockerfile`, `.dockerignore`, `railway.json`, `firebase.json`, `.firebaserc`, `.env.example`, `.gitignore`
- `.github/workflows/ci.yml`, `.github/workflows/ef-migrations.yml`
- `scripts/` — smoke test, application-URL helper, PDF build and CI helpers
- `FirebaseLanding/` — the static presentation site deployed to Firebase Hosting
- `docs/` — all project documentation and the generated PDFs

3. How the work was organised

Working alone, the discipline a group normally supplies had to be built into the process. The practices below were followed throughout.

Practice	How it was applied
Plan before building	Requirements were extracted from the brief into a requirement list before any code was written, and each one was later traced to its implementation in REQUIREMENT_TRACEABILITY.md
Small, single-purpose commits	Work was committed in reviewable steps with messages explaining <i>why</i> , not just what
The build must stay green	<code>dotnet build</code> runs with <code>-p:TreatWarningsAsErrors=true</code> , so no warning is carried forward
Tests alongside features	Every service and controller change came with tests; the suite stands at 211 automated tests
Automated quality gates	Three CI jobs — build and test, the container with its volume proof, and the browser tests — had to pass before work was considered finished
Review against the brief	The traceability document was re-read at the end and every claim checked against the code
No secrets in Git	Demo passwords are generated into a git-ignored file locally and supplied as environment variables in production
Definition of done	Builds, tests pass, CI green, no secrets committed, documentation updated

With no second reviewer, correctness was established by evidence rather than opinion: automated tests, the end-to-end journey checks, the container job that runs the image the way the host runs it, and a final audit in which each requirement was verified against the running system and its database.

4. Individual contribution statement

To be completed in the first person before submission — roughly 100–150 words covering what was built, the hardest problem solved, one thing that would be improved, and links to representative commits.

The hardest problem is worth naming here as a prompt, because it was a genuine one. The application passed every test locally but returned HTTP 500 on every page containing a form once it ran inside a container. The cause was that `KnownIPNetworks = { }` inside an object initialiser adds nothing rather than clearing the default list, so the loopback-only proxy allow-list survived, `X-Forwarded-Proto` was ignored, the request looked insecure and the antiforgery system refused to issue its cookie. It reproduced only from a non-loopback address, which is why local testing never showed it — the container job in CI caught it instead.

5. Marking

With a single author there is no peer assessment to record: the whole of the group mark is attributable to the member named above.